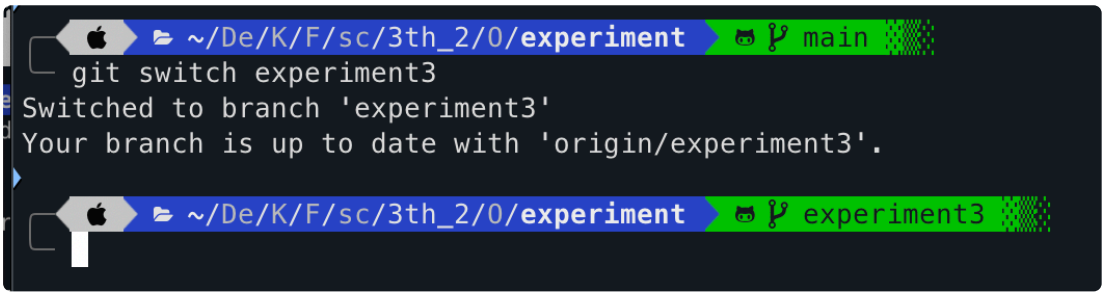
2026 春《操作系统》课程实验报告

实验 3

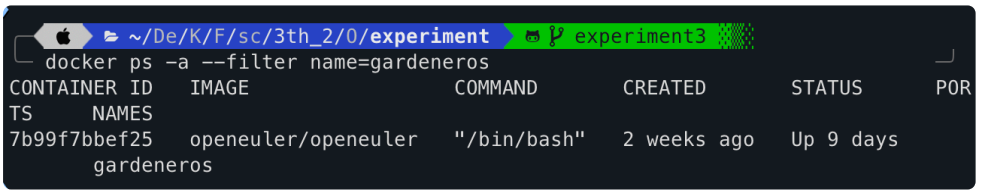
23301037 客昱阳

一.准备工作

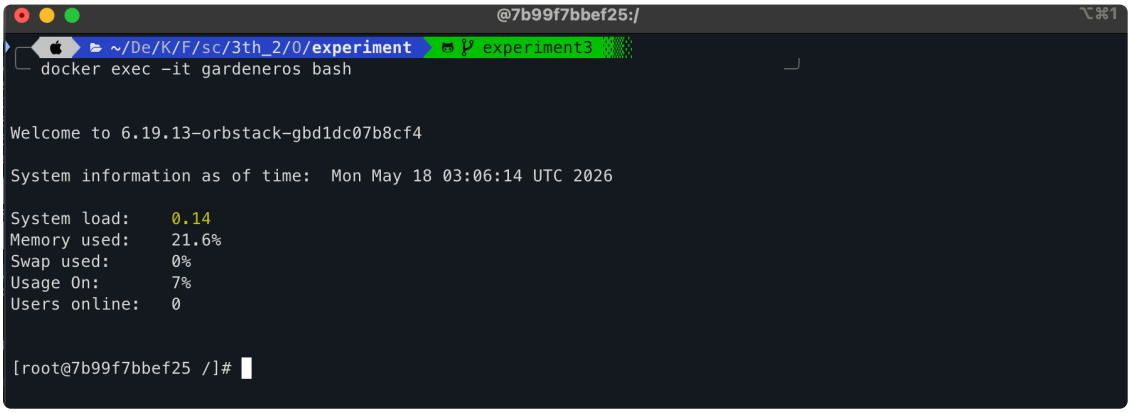
- 将分支切换到实验三所需的 `experiment3` 分支



- 检查一下上次环境配置的 docker 容器已经启动



- 进入容器



二.实验步骤

1.设计和实现应用程序

- 创建用户程序目录 `user` ，后续用户库、系统调用封装以及用户态应用程序都放在这个目录下。

Bash

```
cargo new user_lib
mv user_lib user
rm user/src/main.rs
```

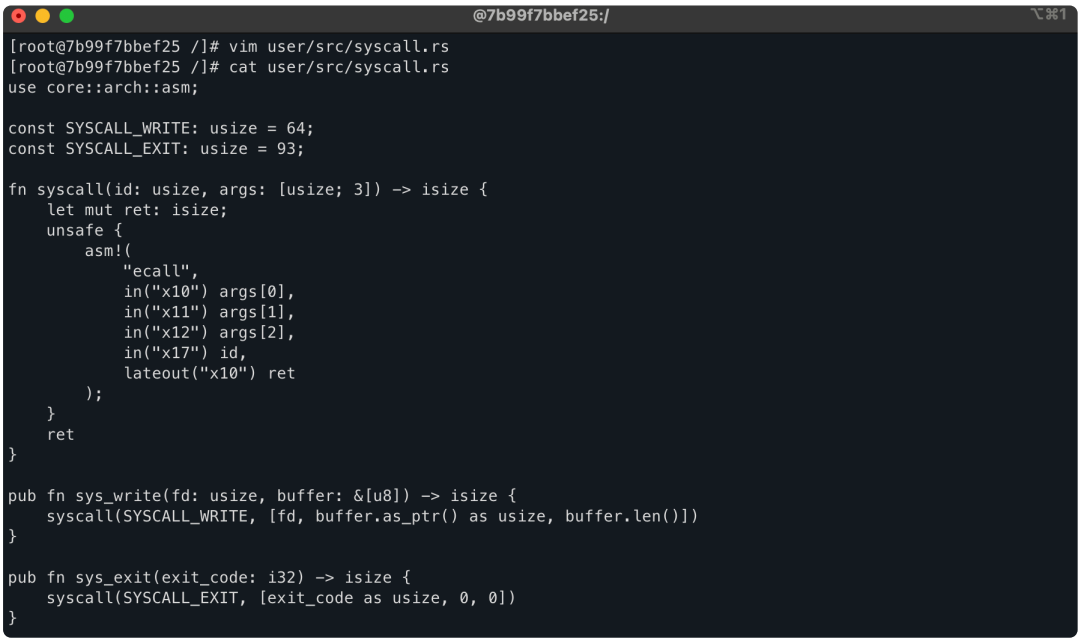
（1）首先实现应用程序与系统约定的两个系统调用 sys_write 和 sys_exit

- 创建 `syscall.rs` ，封装用户态程序需要使用的两个系统调用。 `sys_write` 用来向内核请求输出， `sys_exit` 用来通知内核当前应用程序退出。

Bash

```
vim user/src/syscall.rs
```

写入以下内容：

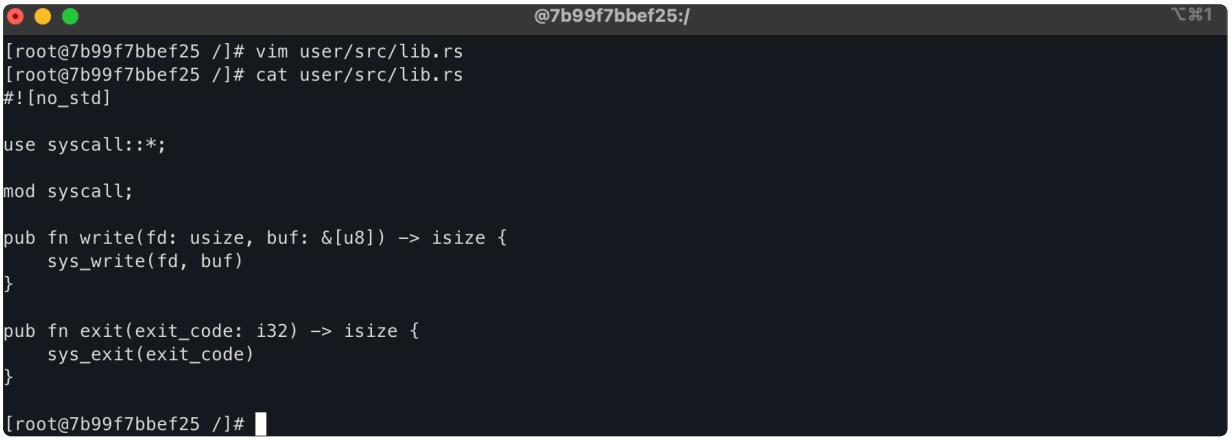


- 修改 `lib.rs` ，把刚才实现的 `sys_write` 和 `sys_exit` 再封装成用户程序可以直接调用的 `write` 和 `exit` 接口。

Bash

```
1 vim user/src/lib.rs
```

写入以下内容：



(2) 实现格式化输出

• 创建 `console.rs` ，基于刚才封装的 `write` 接口实现用户态的 `print!` 和 `println!` 宏。这里传入的文件描述符为 1，表示标准输出。

Bash

```
1 vim user/src/console.rs
```

写入以下内容：

```
[root@7b99f7bbef25 /]# cat user/src/console.rs
use core::fmt::{self, Write};
use super::write;
const STDOUT: usize = 1;
struct Stdout;
impl Write for Stdout {
    fn write_str(&mut self, s: &str) -> fmt::Result {
        write(STDOUT, s.as_bytes());
        Ok(())
    }
}
pub fn print(args: fmt::Arguments) {
    Stdout.write_fmt(args).unwrap();
}
#[macro_export]
macro_rules! print {
    ($fmt: literal $(, $($arg: tt)+)?) => {
        $crate::console::print(format_args!($fmt $(, $($arg)+)?));
    }
}
#[macro_export]
macro_rules! println {
    ($fmt: literal $(, $($arg: tt)+)?) => {
        $crate::console::print(format_args!(concat!($fmt, "\n") $(, $($arg)+)?));
    }
}
```

（3）实现语义支持

- 创建 `lang_items.rs` ，在 `no_std` 环境下实现用户程序的 `panic_handler` 。当用户程序发生 panic 时，输出 panic 的位置和信息

```bash
vim user/src/lang_items.rs
```

写入以下内容：

```
[root@7b99f7bbef25 /]# vim user/src/lang_items.rs
[root@7b99f7bbef25 /]# cat user/src/lang_items.rs
use core::panic::PanicInfo;

#[panic_handler]
fn panic_handler(panic_info: &PanicInfo) -> ! {
    let msg = panic_info.message().as_str().unwrap_or("(no message)");
    if let Some(location) = panic_info.location() {
        println!("Panicked at {}:{}, {}", location.file(), location.line(), msg);
    } else {
        println!("Panicked: {}", msg);
    }
    loop {}
}
```

（4）应用程序内存布局

- 创建 `linker.ld` ，指定用户程序的入口地址为 `0x80400000` ，并安排用户程序中 `.text` 、 `.rodata` 、 `.data` 和 `.bss` 等段的内存布局。

```bash
vim user/src/linker.ld
```

写入以下内容：

```
[root@7b99f7bbef25 /]# vim user/src/linker.ld
[root@7b99f7bbef25 /]# cat user/src/linker.ld
OUTPUT_ARCH(riscv)
ENTRY(_start)
BASE_ADDRESS = 0x80400000;

SECTIONS
{
    . = BASE_ADDRESS;
    .text : {
        *(.text.entry)
        *(.text .text.*)
    }
    .rodata : {
        *(.rodata .rodata.*)
        *(.srodata .srodata.*)
    }
    .data : {
        *(.data .data.*)
        *(.sdata .sdata.*)
    }
    .bss : {
        start_bss = .;
        *(.bss .bss.*)
        *(.sbss .sbss.*)
        end_bss = .;
    }
    /DISCARD/ : {
        *(.eh_frame)
        *(.debug*)
    }
}
```

- 创建 Cargo 配置文件，让 `user` 工程默认编译到 RISC-V 裸机目标，并使用刚才写好的 `src/linker.ld` 作为链接脚本。

Bash

```
mkdir -p user/.cargo
vim user/.cargo/config.toml
```

写入以下内容：

```
[root@7b99f7bbef25 /]# mkdir -p user/.cargo
vim user/.cargo/config.toml
[root@7b99f7bbef25 /]# cat user/.cargo/config.toml
[build]
target = "riscv64gc-unknown-none-elf"

[target.riscv64gc-unknown-none-elf]
rustflags = [
    "-Clink-args=-Tsrc/linker.ld",
]
```

（5）最终形成运行时库 lib.rs

修改 `lib.rs` ，加入用户库入口 `_start` 。用户程序启动后会先清空 `.bss` 段，再调用应用程序自己的 `main` 函数，最后通过 `exit` 系统调用退出。

Bash

```
vim user/src/lib.rs
```

将文件修改为以下内容：

```
[root@7b99f7bbef25 /]# vim user/src/lib.rs
[root@7b99f7bbef25 /]# cat user/src/lib.rs
#![no_std]
#![feature(linkage)]

#[macro_use]
pub mod console;
mod syscall;
mod lang_items;
use syscall::*;

pub fn write(fd: usize, buf: &[u8]) -> isize {
    sys_write(fd, buf)
}

pub fn exit(exit_code: i32) -> isize {
    sys_exit(exit_code)
}

fn clear_bss() {
    unsafe extern "C" {
        fn start_bss();
        fn end_bss();
    }
    let start_bss_ptr = start_bss as *const () as usize;
    let end_bss_ptr = end_bss as *const () as usize;
    (start_bss_ptr..end_bss_ptr).for_each(|a| unsafe { (a as *mut u8).write_volatile(0) });
}
```

```
#[unsafe(no_mangle)]
#[unsafe(link_section = ".text.entry")]
pub extern "C" fn _start() -> ! {
    clear_bss();
    exit(main());
    panic!("unreachable after sys_exit!");
}

#[linkage = "weak"]
#[unsafe(no_mangle)]
fn main() -> i32 {
    panic!("Cannot find main!");
}
```

（6）实现多个不同的应用程序

在 `bin` 下实现多个不同的应用程序

- 先输出 `Hello, world!`，然后执行 `sret` 特权指令的程序 0

Bash

```
1 mkdir -p user/src/bin
2 vim user/src/bin/00hello_world.rs
```

写入以下内容：

- 进行非法写操作的程序 1，用来测试内核能否捕获用户程序的访存异常，并切换到下一个应用程序。

Bash

```
1 vim user/src/bin/01store_fault.rs
```

写入以下内容：

```
@7b99f7bbef25:/
[root@7b99f7bbef25 /]# vim user/src/bin/01store_fault.rs
[root@7b99f7bbef25 /]# cat user/src/bin/01store_fault.rs
#![no_std]
#![no_main]
#[macro_use]
extern crate user_lib;

#[unsafe(no_mangle)]
fn main() -> i32 {
    println!("Into Test store_fault, we will insert an invalid store operation...");
    println!("Kernel should kill this application!");
    unsafe {
        (0x0 as *mut u8).write_volatile(0);
    }
    0
}
```

- 进行多次幂运算并定期输出结果的程序 2，测试普通用户程序能够正常运行并最终退出。

Bash

```
1 vim user/src/bin/02power.rs
```

写入以下内容：

```
@7b99f7bbef25:/
[root@7b99f7bbef25 /]# vim user/src/bin/02power.rs
[root@7b99f7bbef25 /]# cat user/src/bin/02power.rs
#![no_std]
#![no_main]
#[macro_use]
extern crate user_lib;
const SIZE: usize = 10;
const P: u32 = 3;
const STEP: usize = 100000;
const MOD: u32 = 10007;
#[unsafe(no_mangle)]
fn main() -> i32 {
    let mut pow = [0u32; SIZE];
    let mut index: usize = 0;
    pow[index] = 1;
    for i in 1..=STEP {
        let last = pow[index];
        index = (index + 1) % SIZE;
        pow[index] = last * P % MOD;
        if i % 10000 == 0 {
            println!("{}^{}={}", P, i, pow[index]);
        }
    }
    println!("Test power OK!");
    0
}
```

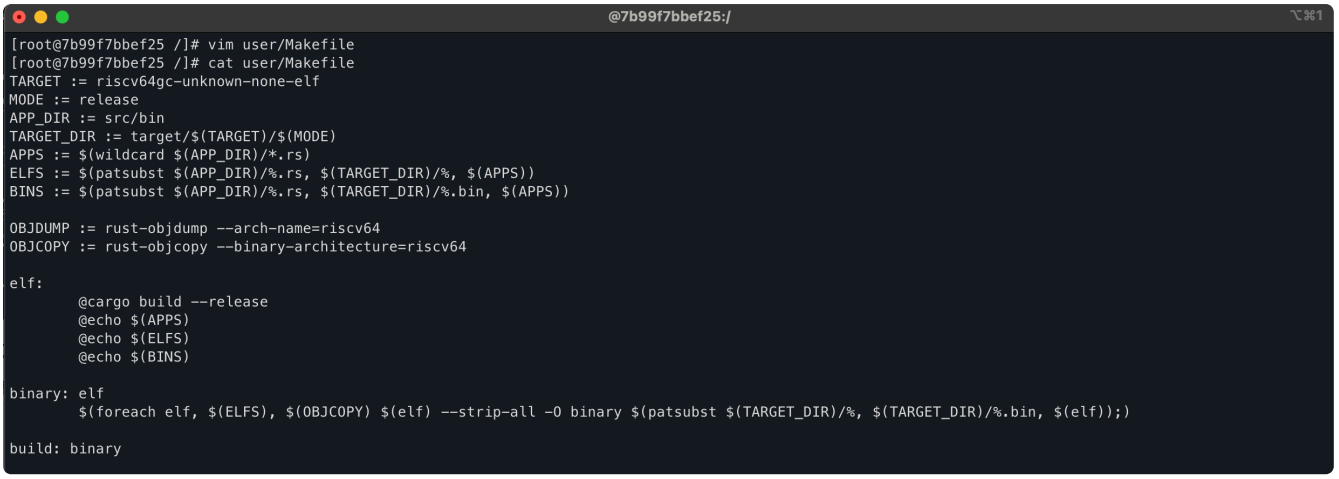
（7）编译生成应用程序二进制码

- 为 `user` 目录编写 `Makefile`，把多个用户应用程序统一编译成 ELF 文件，并进一步转换成内核后续可以加载的 `.bin` 二进制文件。

Bash

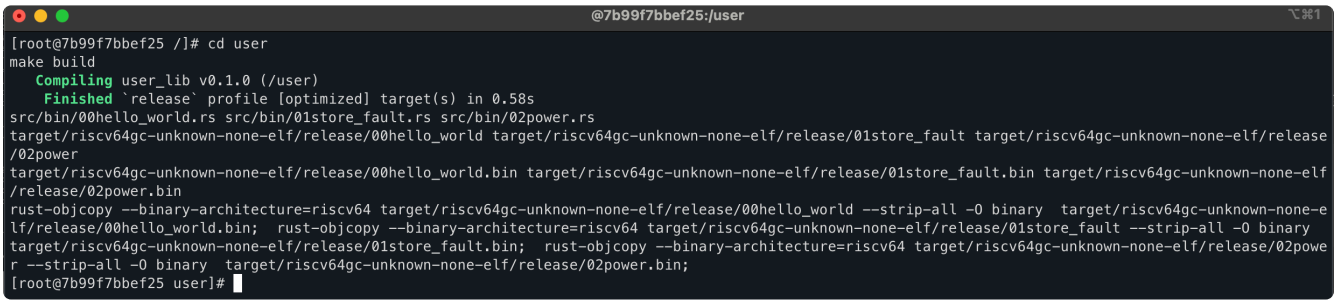
```
1 vim user/Makefile
```

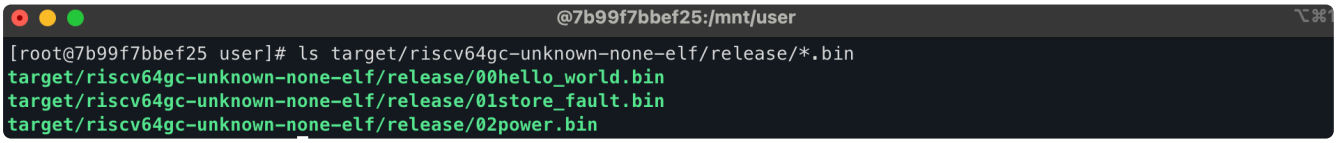
写入以下内容：



- 编译刚才创建的三个用户态应用程序

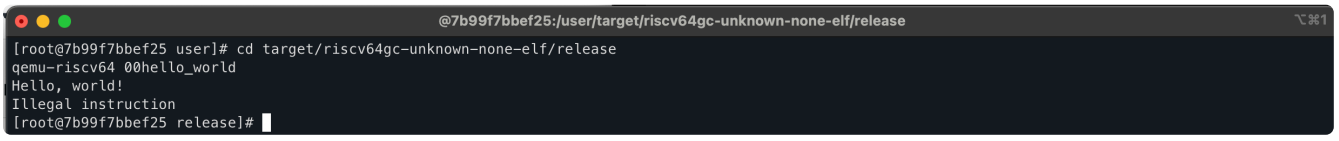
```bash
cd user
make build
```





- 测试我们编写的 `Hello World` 程序

```bash
cd target/riscv64gc-unknown-none-elf/release
qemu-riscv64 00hello_world
```

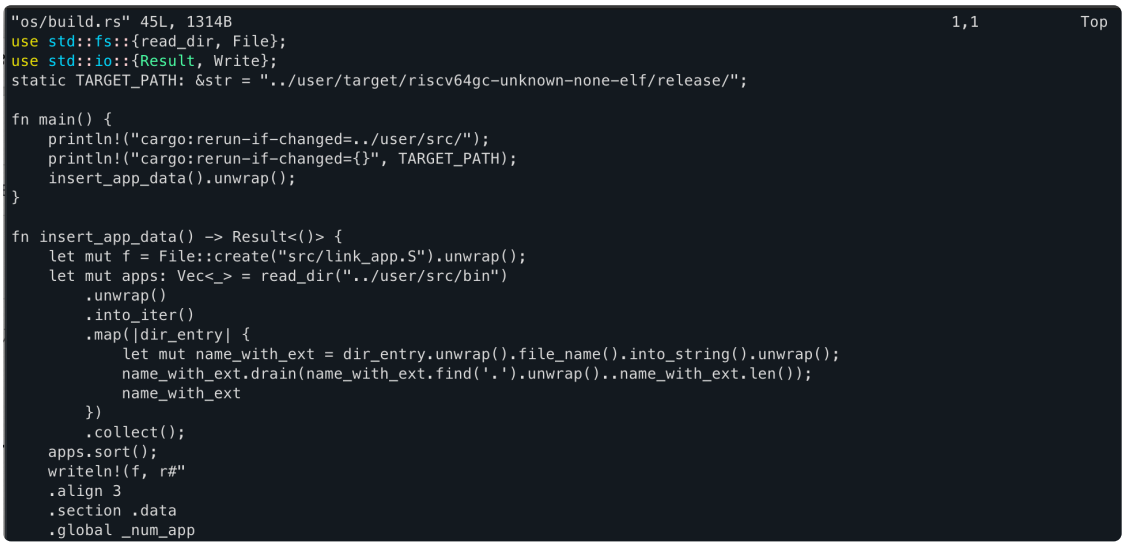


2.链接应用程序到内核

Bash

```
cd /mnt
vim os/build.rs
```

写入以下内容：



```
"os/build.rs" 45L, 1314B                                                    1,1           Top
use std::fs::{read_dir, File};
use std::io::{Result, Write};
static TARGET_PATH: &str = "../user/target/riscv64gc-unknown-none-elf/release/";

fn main() {
    println!("cargo:rerun-if-changed=../user/src/");
    println!("cargo:rerun-if-changed={}", TARGET_PATH);
    insert_app_data().unwrap();
}

fn insert_app_data() -> Result<()> {
    let mut f = File::create("src/link_app.S").unwrap();
    let mut apps: Vec<_> = read_dir("../user/src/bin")
        .unwrap()
        .into_iter()
        .map(|dir_entry| {
            let mut name_with_ext = dir_entry.unwrap().file_name().into_string().unwrap();
            name_with_ext.drain(name_with_ext.find('.').unwrap()..name_with_ext.len());
            name_with_ext
        })
        .collect();
    apps.sort();
    writeln!(f, r#"
    .align 3
    .section .data
    .global _num_app
```

```
_num_app:
    .quad {}"#, apps.len())?;
    for i in 0..apps.len() {
        writeln!(f, r#"    .quad app_{}_start"#, i)?;
    }
    writeln!(f, r#"    .quad app_{}_end"#, apps.len() - 1)?;
    for (idx, app) in apps.iter().enumerate() {
        println!("app_{}: {}", idx, app);
        writeln!(f, r#"
    .section .data
    .global app_{0}_start
    .global app_{0}_end
app_{0}_start:
    .incbin "{2}{1}.bin"
app_{0}_end:"#, idx, app, TARGET_PATH)?;
    }
    Ok(())
}
```

3.找到并加载应用程序二进制码

- 创建 `batch.rs`，用于管理内核中链接进来的多个应用程序。它会记录应用数量、每个应用的起止位置，并负责把当前应用复制到 `0x80400000`，为后续批处理运行做准备。

Bash

```
cd /mnt
vim os/src/batch.rs
```

写入以下内容：

```
@7b99f7bbef25:/mnt                                                    ⌥⌘1
[root@7b99f7bbef25 mnt]# cd /mnt
vim os/src/batch.rs
[root@7b99f7bbef25 mnt]# cat os/src/batch.rs
use core::arch::asm;
use core::cell::RefCell;
use lazy_static::*;
use crate::trap::TrapContext;

const MAX_APP_NUM: usize = 16;
const APP_BASE_ADDRESS: usize = 0x80400000;
const APP_SIZE_LIMIT: usize = 0x20000;

struct AppManager {
    inner: RefCell<AppManagerInner>,
}

struct AppManagerInner {
    num_app: usize,
    current_app: usize,
    app_start: [usize; MAX_APP_NUM + 1],
}

unsafe impl Sync for AppManager {}
```

```
impl AppManagerInner {
    pub fn print_app_info(&self) {
        println!("[kernel] num_app = {}", self.num_app);
        for i in 0..self.num_app {
            println!("[kernel] app_{} [{:#x}, {:#x})", i, self.app_start[i], self.app_start[i + 1]);
        }
    }

    unsafe fn load_app(&self, app_id: usize) {
        if app_id >= self.num_app {
            panic!("All applications completed!");
        }
        println!("[kernel] Loading app_{}", app_id);
        unsafe {
            asm!("fence.i");
            (APP_BASE_ADDRESS..APP_BASE_ADDRESS + APP_SIZE_LIMIT).for_each(|addr| {
                (addr as *mut u8).write_volatile(0);
            });
            let app_src = core::slice::from_raw_parts(self.app_start[app_id] as *const u8, self.app_start[app_id + 1] -
self.app_start[app_id]);
            let app_dst = core::slice::from_raw_parts_mut(APP_BASE_ADDRESS as *mut u8, app_src.len());
            app_dst.copy_from_slice(app_src);
        }
    }

    pub fn get_current_app(&self) -> usize {
        self.current_app
    }

    pub fn move_to_next_app(&mut self) {
        self.current_app += 1;
    }
}
```

```
lazy_static! {
    static ref APP_MANAGER: AppManager = AppManager {
        inner: RefCell::new({
            unsafe extern "C" {
                fn _num_app();
            }
            let num_app_ptr = _num_app as *const () as *const usize;
            let num_app = unsafe { num_app_ptr.read_volatile() };
            let mut app_start: [usize; MAX_APP_NUM + 1] = [0; MAX_APP_NUM + 1];
            let app_start_raw: &[usize] = unsafe {
                core::slice::from_raw_parts(num_app_ptr.add(1), num_app + 1)
            };
            app_start[..=num_app].copy_from_slice(app_start_raw);
            AppManagerInner {
                num_app,
                current_app: 0,
                app_start,
            }
        }),
    };
}
```

```
pub fn init() {
    print_app_info();
}

pub fn print_app_info() {
    APP_MANAGER.inner.borrow().print_app_info();
}

pub fn run_next_app() -> ! {
    let current_app = APP_MANAGER.inner.borrow().get_current_app();
    unsafe {
        APP_MANAGER.inner.borrow().load_app(current_app);
    }
    APP_MANAGER.inner.borrow_mut().move_to_next_app();
    unsafe extern "C" {
        fn __restore(cx_addr: usize);
    }
    unsafe {
        __restore(KERNEL_STACK.push_context(
            TrapContext::app_init_context(APP_BASE_ADDRESS, USER_STACK.get_sp())
        ) as *const _ as usize);
    }
    panic!("Unreachable in batch::run_current_app!");
}
```

- 由于 `batch.rs` 中使用了 `lazy_static!` 宏，需要在 os/Cargo.toml 的 `[dependencies]` 下加入 `lazy_static` 依赖，用于支持全局变量的运行时初始化。

Bash

```
1 vim os/Cargo.toml
```

将文件修改为以下内容：

```
[root@7b99f7bbef25 mnt]# vim os/Cargo.toml
[root@7b99f7bbef25 mnt]# cat os/Cargo.toml
[package]
name = "os"
version = "0.1.0"
edition = "2024"

[dependencies]
lazy_static = { version = "1.4.0", features = ["spin_no_std"] }
[root@7b99f7bbef25 mnt]#
```

4.实现用户栈和内核栈

- 在 `batch.rs` 中继续加入用户栈和内核栈。内核栈用于保存 Trap 上下文，用户栈用于应用程序在用户态运行时使用。RISC-V 的栈向低地址增长，所以取栈顶时需要用数组起始地址加上栈大小。

Bash

```
1 vim os/src/batch.rs
```

添加以下内容：

```
const USER_STACK_SIZE: usize = 4096 * 2;
const KERNEL_STACK_SIZE: usize = 4096 * 2;

#[repr(align(4096))]
struct KernelStack {
    data: [u8; KERNEL_STACK_SIZE],
}

#[repr(align(4096))]
struct UserStack {
    data: [u8; USER_STACK_SIZE],
}

static KERNEL_STACK: KernelStack = KernelStack {
    data: [0; KERNEL_STACK_SIZE],
};

static USER_STACK: UserStack = UserStack {
    data: [0; USER_STACK_SIZE],
};
```

```
impl KernelStack {
    fn get_sp(&self) -> usize {
        self.data.as_ptr() as usize + KERNEL_STACK_SIZE
    }

    pub fn push_context(&self, cx: TrapContext) -> &'static mut TrapContext {
        let cx_ptr = (self.get_sp() - core::mem::size_of::<TrapContext>()) as *mut TrapContext;
        unsafe {
            *cx_ptr = cx;
        }
        unsafe {
            cx_ptr.as_mut().unwrap()
        }
    }
}

impl UserStack {
    fn get_sp(&self) -> usize {
        self.data.as_ptr() as usize + USER_STACK_SIZE
    }
}
```

- 创建 `context.rs`，定义 `TrapContext` 结构体。该结构体用于保存应用程序发生 Trap 时的通用寄存器、 `sstatus` 和 `sepc`，后续汇编代码会按照这个结构保存和恢复上下文。

Bash

```
1 mkdir -p os/src/trap
2 vim os/src/trap/context.rs
```

写入以下内容：

```
@7b99f7bbef25:/mnt
[root@7b99f7bbef25 mnt]# mkdir -p os/src/trap
vim os/src/trap/context.rs
[root@7b99f7bbef25 mnt]# cat os/src/trap/context.rs
use riscv::register::sstatus::Sstatus;

#[repr(C)]
pub struct TrapContext {
    pub x: [usize; 32],
    pub sstatus: Sstatus,
    pub sepc: usize,
}
```

5.实现 trap 管理

- 创建 `trap` 模块，并一次性写入 Trap 初始化和 Trap 分发处理逻辑。 `init()` 用于设置 `stvec`， `trap_handler()` 用于根据异常原因处理系统调用、访存错误和非法指令。

Bash

```
1 vim os/src/trap/mod.rs
```

写入以下内容：

```
[root@7b99f7bbef25 mnt]# vim os/src/trap/mod.rs
[root@7b99f7bbef25 mnt]# cat os/src/trap/mod.rs
mod context;

use core::arch::global_asm;
use riscv::register::{
    mtvec::TrapMode,
    stvec,
    scause::{self, Exception, Trap},
    stval,
};
use crate::batch::run_next_app;
use crate::syscall::syscall;

global_asm!(include_str!("trap.S"));

pub fn init() {
    unsafe extern "C" {
        fn __alltraps();
    }
    unsafe {
        stvec::write(__alltraps as *const () as usize, TrapMode::Direct);
    }
}
```

```
#[unsafe(no_mangle)]
pub fn trap_handler(cx: &mut TrapContext) -> &mut TrapContext {
    let scause = scause::read();
    let stval = stval::read();
    match scause.cause() {
        Trap::Exception(Exception::UserEnvCall) => {
            cx.sepc += 4;
            cx.x[10] = syscall(cx.x[17], [cx.x[10], cx.x[11], cx.x[12]]) as usize;
        }
        Trap::Exception(Exception::StoreFault) | Trap::Exception(Exception::StorePageFault) => {
            println!("[kernel] PageFault in application, core dumped.");
            run_next_app();
        }
        Trap::Exception(Exception::IllegalInstruction) => {
            println!("[kernel] IllegalInstruction in application, core dumped.");
            run_next_app();
        }
        _ => {
            panic!("Unsupported trap {:?}, stval = {:#x}!", scause.cause(), stval);
        }
    }
    cx
}

pub use context::TrapContext;
```

- 由于 trap 模块中使用了 `riscv` 库读取和修改 CSR 寄存器，所以需要在 `os/Cargo.toml` 中继续加入 `riscv` 依赖。

Bash

```
1 vim os/Cargo.toml
```

将文件修改为以下内容：

```
@7b99f7bbef25:/mnt
[root@7b99f7bbef25 mnt]# vim os/Cargo.toml
[root@7b99f7bbef25 mnt]# cat os/Cargo.toml
[package]
name = "os"
version = "0.1.0"
edition = "2024"

[dependencies]
lazy_static = { version = "1.4.0", features = ["spin_no_std"] }
riscv = { git = "https://github.com/rcore-os/riscv", features = ["inline-asm"] }
[root@7b99f7bbef25 mnt]#
```

- 创建 trap.S，实现 `__alltraps` 和 `__restore`。`__alltraps` 负责在进入内核时保存用户程序上下文，`__restore` 负责恢复上下文并通过 sret 返回用户态。

Bash

```
vim os/src/trap/trap.S
```

```
@7b99f7bbef25:/mnt
[root@7b99f7bbef25 mnt]# vim os/src/trap/trap.S
[root@7b99f7bbef25 mnt]# cat os/src/trap/trap.S
.altmacro

.macro SAVE_GP n
    sd x\n, \n*8(sp)
.endm

.macro LOAD_GP n
    ld x\n, \n*8(sp)
.endm

    .section .text
    .globl __alltraps
    .globl __restore
    .align 2
```

```
__alltraps:
    csrrw sp, sscratch, sp
    addi sp, sp, -34*8
    sd x1, 1*8(sp)
    sd x3, 3*8(sp)
    .set n, 5
    .rept 27
        SAVE_GP %n
        .set n, n+1
    .endr
    csrr t0, sstatus
    csrr t1, sepc
    sd t0, 32*8(sp)
    sd t1, 33*8(sp)
    csrr t2, sscratch
    sd t2, 2*8(sp)
    mv a0, sp
    call trap_handler
```

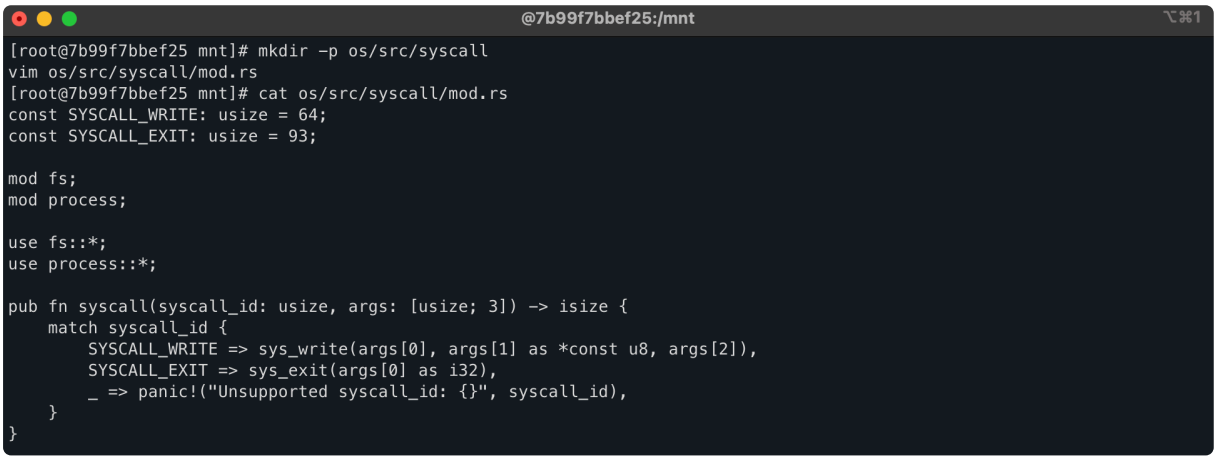
```
__restore:
    mv sp, a0
    ld t0, 32*8(sp)
    ld t1, 33*8(sp)
    ld t2, 2*8(sp)
    csrw sstatus, t0
    csrw sepc, t1
    csrw sscratch, t2
    ld x1, 1*8(sp)
    ld x3, 3*8(sp)
    .set n, 5
    .rept 27
        LOAD_GP %n
        .set n, n+1
    .endr
    addi sp, sp, 34*8
    csrrw sp, sscratch, sp
    sret
```

- 创建 `syscall` 模块。`syscall` 函数根据系统调用编号进行分发，`sys_write` 负责输出用户程序传来的字符串，`sys_exit` 负责结束当前应用并运行下一个应用。

Bash

```
1 mkdir -p os/src/syscall
2 vim os/src/syscall/mod.rs
```

写入以下内容：



- 创建 `fs.rs`，实现 `sys_write` 系统调用。当前只支持文件描述符 1，也就是标准输出，用来把用户程序传来的字符串打印到屏幕。

Bash

```
1 vim os/src/syscall/fs.rs
```

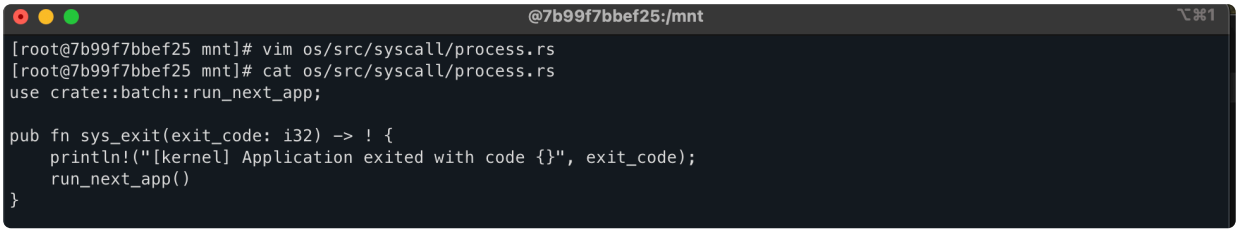
写入以下内容：

- 创建 `process.rs`，实现 `sys_exit` 系统调用。当用户程序调用 `exit` 时，内核输出退出码，然后切换执行下一个应用程序。

Bash

```
1 vim os/src/syscall/process.rs
```

写入以下内容：



6.执行应用程序

为 `TrapContext` 实现 `app_init_context`。内核在启动用户程序前，会先构造一个特殊的 Trap 上下文，设置用户程序入口地址、用户栈地址，并把 `sstatus` 中的 `SPP` 设置为 `User`，这样 `__restore` 执行 `sret` 后就会进入用户态。

Bash

```
1 vim os/src/trap/context.rs
```

追加以下内容

```
impl TrapContext {
    pub fn set_sp(&mut self, sp: usize) {
        self.x[2] = sp;
    }

    pub fn app_init_context(entry: usize, sp: usize) -> Self {
        let mut sstatus = sstatus::read();
        sstatus.set_spp(SPP::User);
        let mut cx = Self {
            x: [0; 32],
            sstatus,
            sepc: entry,
        };
        cx.set_sp(sp);
        cx
    }
}
```

7.修改 main.rs

引入新实现的 `syscall`、`trap` 和 `batch` 模块，并把 `link_app.S` 链接进内核。内核启动后先初始化 Trap，再打印应用信息，最后开始批量执行用户程序。

Bash

```
1 vim os/src/main.rs
```

将文件修改为以下内容：

```
@7b99f7bbef25:/mnt
[root@7b99f7bbef25 mnt]# vim os/src/main.rs
[root@7b99f7bbef25 mnt]# cat os/src/main.rs
#![no_std]
#![no_main]

#[macro_use]
mod console;
mod lang_items;
mod sbi;
mod syscall;
mod trap;
mod batch;

use core::arch::global_asm;

global_asm!(include_str!("entry.asm"));
global_asm!(include_str!("link_app.S"));
```

```
fn clear_bss() {
    unsafe extern "C" {
        fn sbss();
        fn ebss();
    }
    let sbss_ptr = sbss as *const () as usize;
    let ebss_ptr = ebss as *const () as usize;
    (sbss_ptr..ebss_ptr).for_each(|a| unsafe { (a as *mut u8).write_volatile(0) });
}

#[unsafe(no_mangle)]
pub fn rust_main() -> ! {
    clear_bss();
    println!("[Kernel] Hello, world!");
    trap::init();
    batch::init();
    batch::run_next_app();
}
```

8.编译运行检查

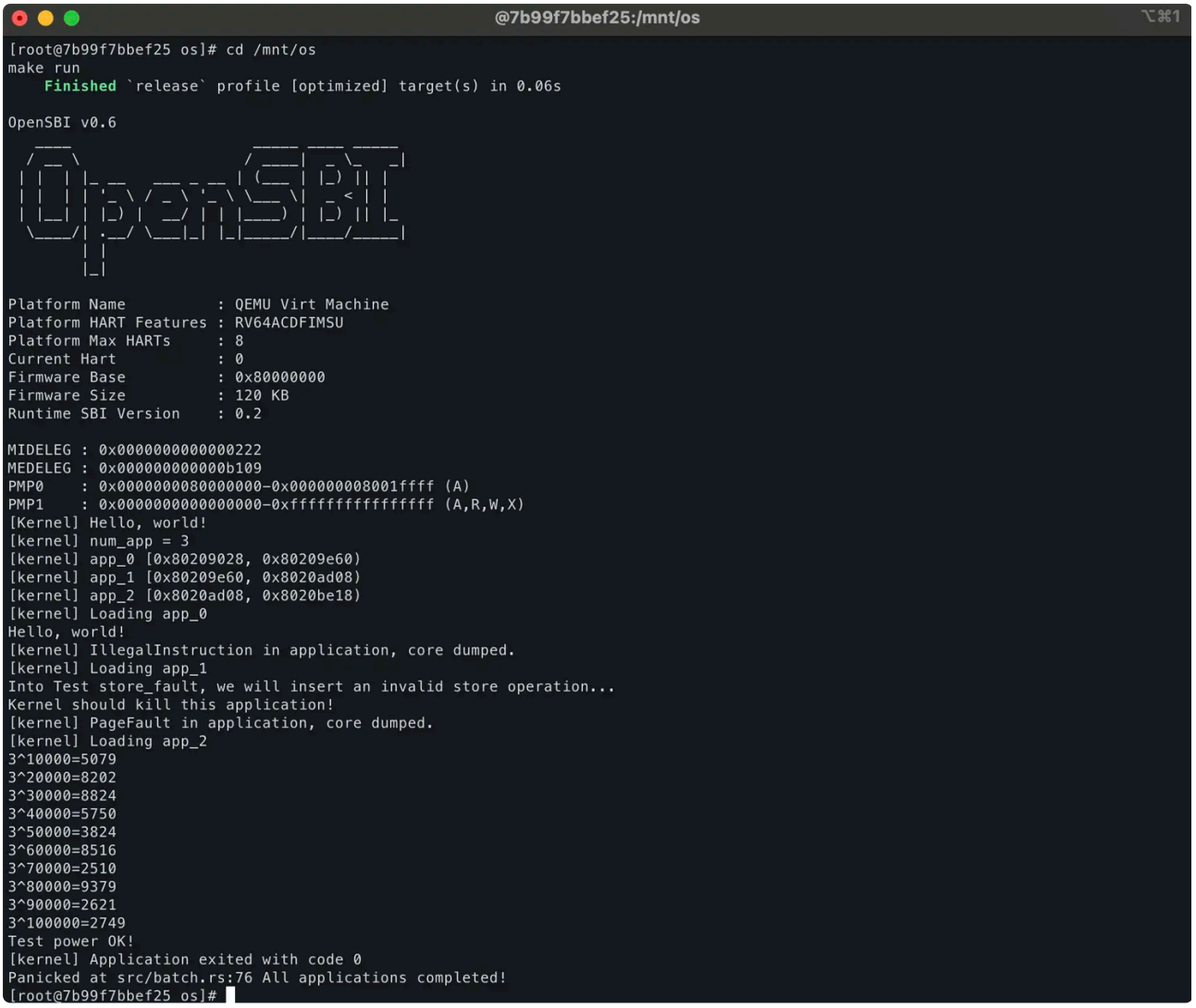
Bash

```
cd /mnt/user
make build
```

```
@7b99f7bbef25:/mnt/user
[root@7b99f7bbef25 mnt]# cd /mnt/user
make build
    Finished `release` profile [optimized] target(s) in 0.01s
src/bin/00hello_world.rs src/bin/01store_fault.rs src/bin/02power.rs
target/riscv64gc-unknown-none-elf/release/00hello_world target/riscv64gc-unknown-none-elf/release/01store_fault target/riscv64gc-unknown-none-elf/release/02power
target/riscv64gc-unknown-none-elf/release/00hello_world.bin target/riscv64gc-unknown-none-elf/release/01store_fault.bin target/riscv64gc-unknown-none-elf/release/02power.bin
rust-objcopy --binary-architecture=riscv64 target/riscv64gc-unknown-none-elf/release/00hello_world --strip-all -O binary  target/riscv64gc-unknown-none-elf/release/00hello_world.bin;  rust-objcopy --binary-architecture=riscv64 target/riscv64gc-unknown-none-elf/release/01store_fault --strip-all -O binary  target/riscv64gc-unknown-none-elf/release/01store_fault.bin;  rust-objcopy --binary-architecture=riscv64 target/riscv64gc-unknown-none-elf/release/02power --strip-all -O binary  target/riscv64gc-unknown-none-elf/release/02power.bin;
[root@7b99f7bbef25 user]#
```

Bash

```
cd /mnt/os
make run
```



三.问题回答

1.分析应用程序的实现过程，并实现一个自己的应用程序；

- **程序实现过程**

(1) 用户程序首先需要通过 `ecall` 发起系统调用。系统调用编号放在 `x17`，参数放在 `x10`、`x11`、`x12`，返回值仍然放在 `x10`。

Rust

```
1 asm!("ecall", in("x10") args[0], in("x11") args[1],
2       in("x12") args[2], in("x17") id, lateout("x10") ret);
```

(2) 在 ecall 的基础上封装输出和退出两个系统调用。用户程序通过 `sys_write` 请求内核输出，通过 `sys_exit` 通知内核程序结束。

Rust

```rust
pub fn sys_write(fd: usize, buffer: &[u8]) -> isize {
    syscall(SYSCALL_WRITE, [fd, buffer.as_ptr() as usize, buffer.len()])
}

pub fn sys_exit(exit_code: i32) -> isize {
    syscall(SYSCALL_EXIT, [exit_code as usize, 0, 0])
}
```

(3) 用户库进一步封装 `write` 和 `exit`，让应用程序不需要直接接触底层系统调用。

Rust

```rust
pub fn write(fd: usize, buf: &[u8]) -> isize {
    sys_write(fd, buf)
}
pub fn exit(exit_code: i32) -> isize {
    sys_exit(exit_code)
}
```

(4) 格式化输出基于 `write` 实现。`println!` 最终会把字符串写到文件描述符 1，也就是标准输出。

Rust

```rust
fn write_str(&mut self, s: &str) -> fmt::Result {
    write(1, s.as_bytes());
    Ok(())
}
```

(5) 用户库定义 `_start` 作为真正入口。它先清空 `.bss`，然后调用应用程序的 `main`，最后通过 `exit` 退出。

Rust

```rust
pub extern "C" fn _start() -> ! {
    clear_bss();
    exit(main());
    panic!("unreachable after sys_exit!");
}
```

(6) 应用程序只需要放在 `user/src/bin` 下，并实现自己的 `main` 函数。以 Hello World 程序为例

Rust

```rust
#![no_std]
#![no_main]
#[macro_use]
extern crate user_lib;

#[unsafe(no_mangle)]
fn main() -> i32 {
    println!("Hello, world!");
    0
}
```

该程序运行时会通过 `println!` 发起 `sys_write` 系统调用，请求内核输出 Hello, world!；当 `main` 返回 0 后，用户库入口 `_start` 会调用 `sys_exit`，通知内核当前程序正常结束。

- **实现自己的应用程序**

自己实现一个打印 1 到 100 之间质数的程序。在 `user/src/bin` 目录下添加 `03prime.rs` 文件，写入以下代码。

Rust

```rust
#![no_std]
#![no_main]
#[macro_use]
extern crate user_lib;
#[unsafe(no_mangle)]
fn main() -> i32 {
    println!("Prime numbers from 1 to 100:");
    for n in 2..=100 {
        let mut is_prime = true;
        let mut d = 2;
        while d * d <= n {
            if n % d == 0 {
                is_prime = false;
                break;
            }
            d += 1;
        }
        if is_prime {
            print!("{} ", n);
        }
    }
    println!("");
    0
}
```

编译并运行程序

Bash

```bash
cd user
make build
cd target/riscv64gc-unknown-none-elf/release
qemu-riscv64 03prime
```

```
Test power OK!
[kernel] Application exited with code 0
[kernel] Loading app_3
Prime numbers from 1 to 100:
2 3 5 7 11 13 17 19 23 29 31 37 41 43 47 53 59 61 67 71 73 79 83 89 97
[kernel] Application exited with code 0
Panicked at src/batch.rs:76 All applications completed!
[root@7b99f7bbef25 os]#
```

2.分析应用程序的链接、加载和执行过程；

(1) 应用程序通过链接脚本固定运行地址。所有用户程序都会被链接到 `0x80400000`，并从 `_start` 入口开始执行。

Rust

```
BASE_ADDRESS = 0x80400000;
.text : {
    *(.text.entry)
    *(.text .text.*)
}
```

(2) 用户程序编译后会被转换成 `.bin` 二进制文件，之后内核构建脚本会把这些应用程序二进制码链接进内核镜像。

Bash

```
app_0_start:
    .incbin "../user/target/riscv64gc-unknown-none-elf/release/00hello_world.bin"
app_0_end:
```

其中 `app_0_start` 和 `app_0_end` 标记了应用程序在内核中的起止位置，内核运行时可以根据这些符号找到对应的应用程序。

(3) 内核运行时通过 `AppManager` 管理这些应用程序。加载应用程序时，内核会把当前应用程序从内核镜像中复制到 `0x80400000`

Rust

```
let app_dst = core::slice::from_raw_parts_mut(APP_BASE_ADDRESS as *mut u8, app_src.len());
app_dst.copy_from_slice(app_src);
```

(4) 应用程序加载完成后，内核构造用户程序初始上下文，并调用 `__restore` 切换到用户态执行。

Rust

```
TrapContext::app_init_context(APP_BASE_ADDRESS, USER_STACK.get_sp())
```

其中 `APP_BASE_ADDRESS` 是应用程序入口地址，`USER_STACK.get_sp()` 是用户栈栈顶。随后 `__restore` 会恢复上下文并通过 `sret` 进入用户态，开始运行应用程序。

3.分析 Trap 是如何实现的?

(1) 内核首先设置 `stvec` 寄存器，让 CPU 在发生 Trap 时跳转到统一的入口 `__alltraps`。

Rust

```rust
pub fn init() {
    unsafe extern "C" {
        fn __alltraps();
    }
    unsafe {
        stvec::write(__alltraps as *const () as usize, TrapMode::Direct);
    }
}
```

这样用户程序执行 `ecall`、非法指令或发生访存错误时，都会先进入 `__alltraps`。

(2) `__alltraps` 负责保存用户程序上下文。进入 Trap 后，先切换到内核栈，再把通用寄存器、`sstatus` 和 `sepc` 保存到 `TrapContext` 中。

Rust

```rust
__alltraps:
    csrrw sp, sscratch, sp
    addi sp, sp, -34*8
    sd x1, 1*8(sp)
    sd x3, 3*8(sp)
    csrr t0, sstatus
    csrr t1, sepc
```

保存上下文后，内核就可以安全地执行 Trap 处理函数。

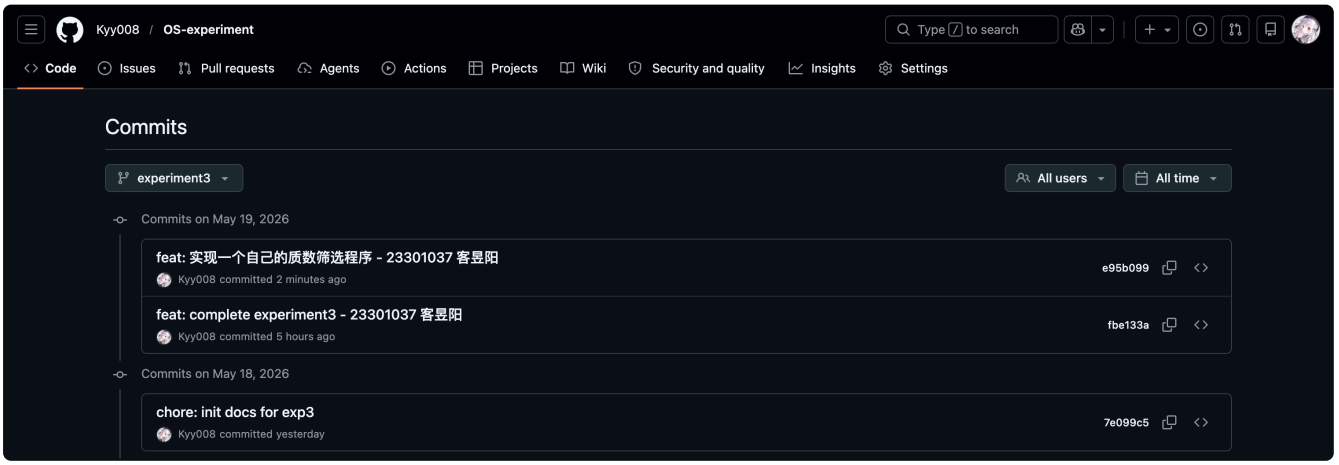
(3) `trap_handler` 根据 `scause` 判断 `Trap` 类型，并进行对应处理。系统调用会分发到 syscall，非法指令和访存错误会结束当前应用并运行下一个应用。

```rust
match scause.cause() {
    Trap::Exception(Exception::UserEnvCall) => {
        cx.sepc += 4;
        cx.x[10] = syscall(cx.x[17], [cx.x[10], cx.x[11], cx.x[12]]) as usize;
    }
    Trap::Exception(Exception::IllegalInstruction) => {
        println!("[kernel] IllegalInstruction in application, core dumped.");
        run_next_app();
    }
    _ => panic!("Unsupported trap!"),
}
```

(4) Trap 处理完成后，通过 `__restore` 恢复上下文，并执行 `sret` 返回用户态继续运行。

```rust
__restore:
    mv sp, a0
    ld t0, 32*8(sp)
    ld t1, 33*8(sp)
    csrw sstatus, t0
    csrw sepc, t1
    sret
```

四.Git 提交截图



五.其他说明

无